



파머완 9장 : 추천 시스템

☼ 상태	진행 중
👤 담당자	 시현 이
📅 마감일	@12/23/2025
🔧 작업 유형	개념정리 및 필사
≡ 설명	파이썬 머신러닝 완벽 가이드_개정2판_제9장 개념 정리+ 필사 링크
🕒 업데이트 시간	@December 24, 2025 3:07 PM

01.추천 시스템의 개요와 배경

추천 시스템의 개요

추천시스템이 개인의 취향을 정확히 이해하고 콘텐츠를 추천하며 전자상거래에서 고객들을 유치하고 있음. 추천시스템은 사용자 자신도 좋아하는지 몰랐던 취향을 시스템이 발견하고 그에 맞는 콘텐츠를 추천함.

⇒ 사이트에 대한 신뢰도가 상승하면 더 많은 데이터가 축적되면서 추천이 다양해지고 다양한 결과를 얻을 수 있는 선순화 시스템이 구축됨

온라인 스토어의 필수 요소, 추천시스템

온라인에서 소비지들에게 상품을 추천함으로써 한정된 시간에 상품을 고를 수 있게 해줌. 상품 선택에 대한 압박감을 추천 시스템이 타개해줌

<추천시스템 구성 요소>

- 사용자가 어떤 상품을 구매했는가
- 사용자가 어떤 상품을 둘러보거나 장바구니에 넣었는가
- 사용자가 평가한 영화 평점은? 제품 평가는?

- 사용자가 스스로 작성한 자신의 취향은?
- 사용자가 무엇을 클릭했는가



추천 시스템의 유형

1. 콘텐츠 기반 필터링
2. 협업 필터링(최근접 이웃 협업 필터링/잠재 요인 협업 필터링)

초창기: 콘텐츠 기반 필터링, 최근접 이웃 협업 필터링

→ 행렬 분해 기법 이용한 잠재 요인 협업 필터링 방식 우세

→ 최근엔 하이브리드 형식으로 콘텐츠 기반 + 협업 기반 결합해 강화

02. 콘텐츠 기반 필터링 추천 시스템

: 사용자가 특정한 아이템을 선호할 경우, 비슷한 콘텐츠를 가진 다른 아이템을 추천

ex) 특정 영화의 평점을 높게 줌 → 그 영화의 장르, 출연 배우, 감독, 영화 키워드 등의 콘텐츠와 유사한 다른 영화 추천

03. 최근접 이웃 협업 필터링

: 사용자가 아이템에 매긴 평점 정보나 상품 구매 이력과 같은 사용자 행동 인식만을 기반으로 추천을 수행하는 것

[주요 목표]

- 사용자-아이템 평점 매트릭스와 같은 축적된 사용자 행동 데이터를 기반으로 사용자가 아직 평가하지 않은 아이템을 예측 평가하는 것.
- 사용자가 평가한 다른 아이템을 기반으로 사용자가 평가하지 않은 아이템의 예측 평가를 도출하는 방식

사용자가 평가하지 않은 아이템을 평가한
아이템에 기반하여 예측 평가하는 알고리즘

	Item 1	Item 2	Item 3	Item 4
User 1	3		3	✓
User 2	4	2		3
User 3		1	2	2

[사용자-아이템 평점 행렬]

- 최근접 이웃 방식과 잠재 요인 방식 모두 사용자-아이템 평점 행렬 데이터에만 의지해 추천 수행
- Row : 개별 사용자/Column: 개별 아이템/열과 행이 만나는 구간: 평점
- 다차원 행렬 + 희소행렬(사용자가 아이템에 대한 평점을 매기는 경우 적음)

로우 레벨 형태의 사용자 -아이템 평점 데이터

User ID	Item ID	Rating
User 1	Item 1	3
User 1	Item 3	3
User 2	Item 1	4
User 2	Item 2	1
User 3	Item 4	5

변환

사용자 로우, 아이템 칼럼으로 구성된 사용자 -아이템 평점 데이터

	Item 1	Item 2	Item 3	Item 4
User 1	3		3	
User 2	4	1		
User 3				5

[최근점 이웃 협업 필터링] (=협업 필터링)

- 사용자 기반(User-User): 사용자와 비슷한 고객들이 구매한 상품 추천
 - 유사한 다른 사용자를 Top-N으로 선정\

		다크 나이트	인터스텔라	엣지 오브 투모로우	프로메테우스	스타워즈 라스트제다이
상호간 유사도 높음	사용자 A	5	4	4		
	사용자 B	5	3	4	5	3
	사용자 C	4	3	3	2	5

사용자 A는 사용자 C보다 사용자 B와 영화 평점 측면에서 유사도가 높음. 따라서 사용자 A에게는 사용자 B가 재미있게 본 '프로메테우스'를 추천

- 아이템 기반(Item-Item): 해당 상품을 선택한 다른 고객들이 구매한 상품들 추천
 - 아이템이 가진 속성과는 상관없음. 사용자들의 그 아이템에 대한 호불호 평가 척도가 유사한 아이템을 추천하는 기준이 됨
 - 사용자 기반과 행과 열이 반대

		사용자 A	사용자 B	사용자 C	사용자 D	사용자 E
상호간 유사도 높음	다크 나이트	5	4	5	5	5
	프로메테우스	5	4	4		5
	스타워즈 라스트제다이	4	3	3		4

여러 사용자들의 평점을 기준으로 볼 때 '다크 나이트'와 가장 유사한 영화는 '프로메테우스'

일반적으로는 사용자 기반보다는 아이템 기반 협업 필터링이 정확도가 더 높음

→ 같은 영화를 좋아한다고 해서 취향이 비슷하다고 판단하기는 어려움

→ 유명한 영화는 대부분 사람들이 관람

→ 평점을 매긴 영화 개수는 적음

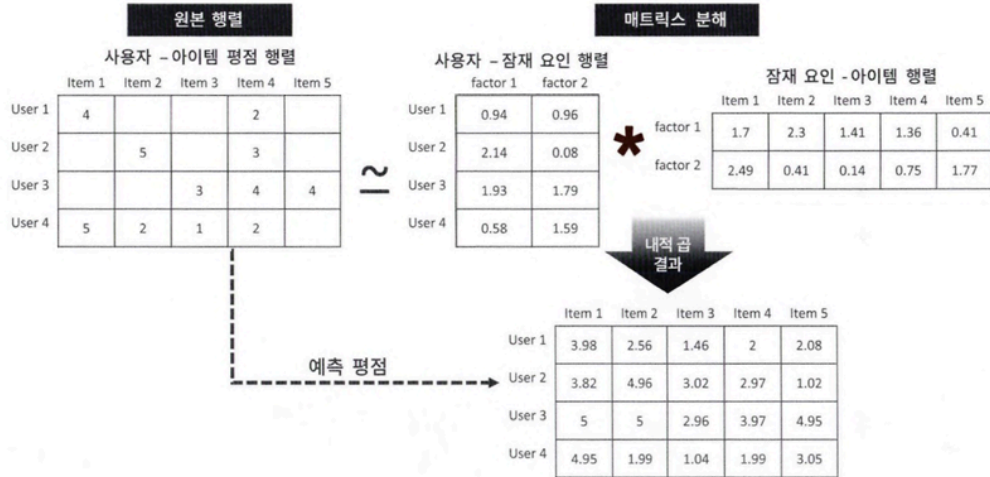
⇒ 최근접 이웃 협업 필터링은 대부분 아이템 기반의 알고리즘을 적용

04. 잠재 요인 협업 필터링

잠재 요인 협업 필터링의 이해

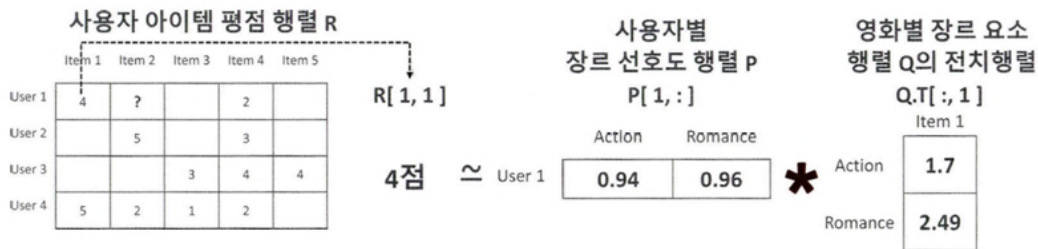
: 사용자-아이템 평점 매트릭스 속에 숨어 있는 잠재 요인을 추출해 추천 예측을 할 수 있게 하는 방법

- 대규모 다차원 행렬을 SVD와 같은 차원 감소 기법으로 분해하는 과정에서 잠재 요인 추출 = 행렬 분해(Matrix Factorization)
- '잠재 요인'이 어떤 것인지는 명확히 정의할 수 없음.
- But. '잠재 요인'을 기반으로 다차원 희소 행렬인 사용자-아이템 행렬 데이터를 저차원 밀집 행렬의 사용자-잠재 요인 행렬과 아이템-잠재 요인 행렬의 전치 행렬(즉, 잠재 요인-아이템 행렬)로 분해할 수 있음
 - 두 행렬의 내적을 통해 새로운 예측 사용자-아이템 평점 행렬 데이터 생성
 - 사용자-잠재 요인 행렬: 사용자의 영화 장르에 대한 선호도/아이템-잠재요인 행렬은 영화의 장르별 특성값으로 정의

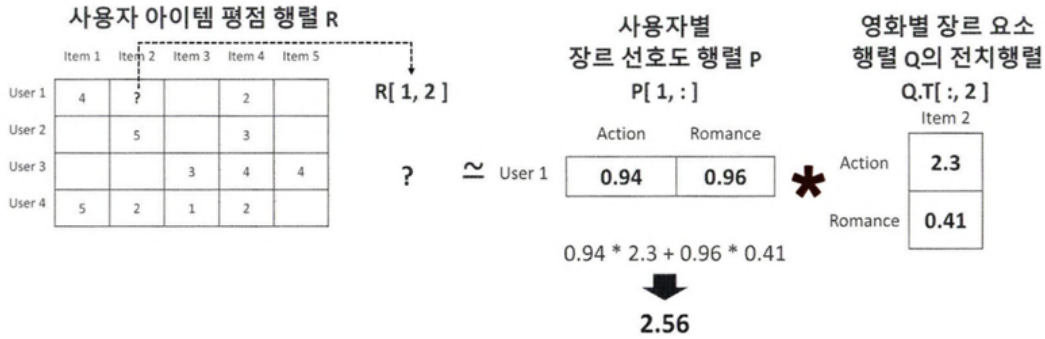


〈 행렬 분해를 통한 잠재 요인 협업 필터링 〉

- 사용자-아이템 평점 행렬 R 에서 사용자(User)의 아이템(Item)에 대한 평점을 $R(u, i)$
- 사용자-잠재요인 행렬을 사용자의 영화 장르별 선호도 행렬 P 로 가정. factor 1을 액션 선호도, factor 2는 로맨스 선호도 $\Rightarrow P(u, k)$ 에서 u 는 사용자 아이디, k 는 잠재 요인 칼럼인 장르별 선호도
- 아이템-잠재 요인 행렬은 영화별로 여러 장르 요소로 구성된 영화의 장르별 요소 행렬 Q 로 가정. $\Rightarrow Q(i, k)$ 에서 i 는 아이템 아이디, k 는 잠재 요인 칼럼인 장르별 요소
- Q 와 P 의 내적 계산을 통해 예측 평점을 계산하기 위해 $Q.T$ 로 변환(전치)



평점: 사용자의 장르별 선호도 벡터 * 영화의 장르별 특성 벡터



행렬 분해의 이해

:다차원의 매트릭스를 저차원 매트릭스로 분해하는 기법-SVD(Singular Vector Decomposition), NMF(Non-Negative Matrix Factorization) 등

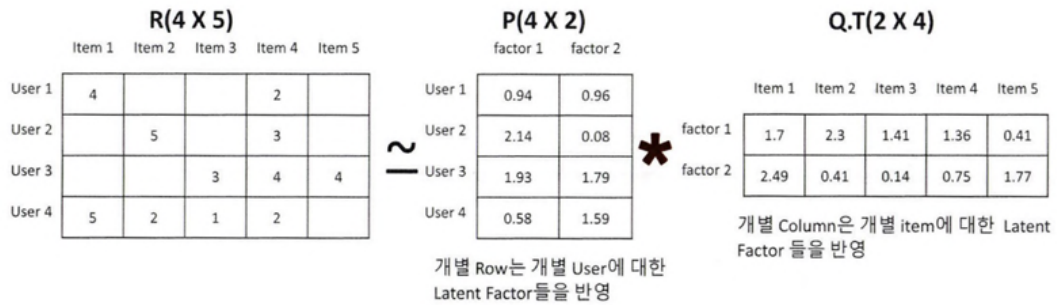
- M개의 사용자 행과 N개의 아이템 열을 가진 평점 행렬 R은 $M \times N$ 차원으로 구성됨. 행렬 분해를 통해서 사용자-K차원 잠재 요인 행렬 P($M \times K$ 차원)와 K 차원 잠재 요인 - 아이템 행렬 Q.T($K \times N$ 차원)로 분해될 수 있음



$$R = P * Q.T$$

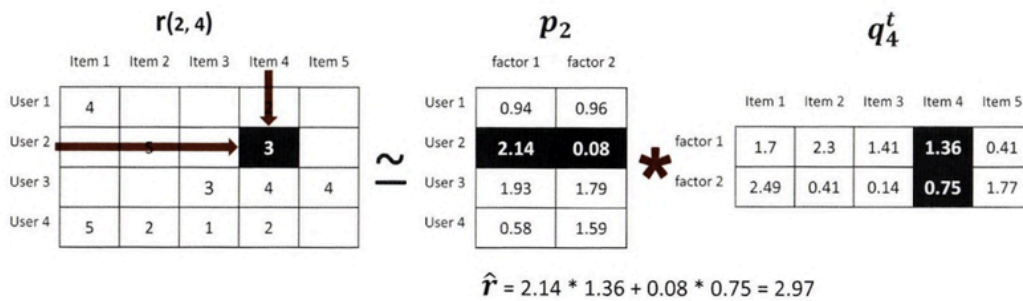
- M: 총 사용자 수
- N: 총 아이템 수
- K: 잠재 요인의 차원 수
- R: $M \times N$ 차원의 사용자-아이템 평점 행렬
- P: 사용자와 잠재 요인과의 관계 값을 가지는 $M \times K$ 차원의 사용자-잠재 요인 행렬
- Q: 아이템과 잠재 요인과의 관계 값을 가지는 $N \times K$ 차원의 아이템-잠재 요인 행렬
- Q.T는 Q 매트릭스의 행과 열 값을 교환한 전치 행렬

ex) 행렬 내에 널(NaN) 값을 많이 가지는 고차원의 희소 행렬인 R행렬은 다음 그림과 같이 저차원의 밀집 행렬인 P 행렬과 Q 행렬로 분해될 수 있음

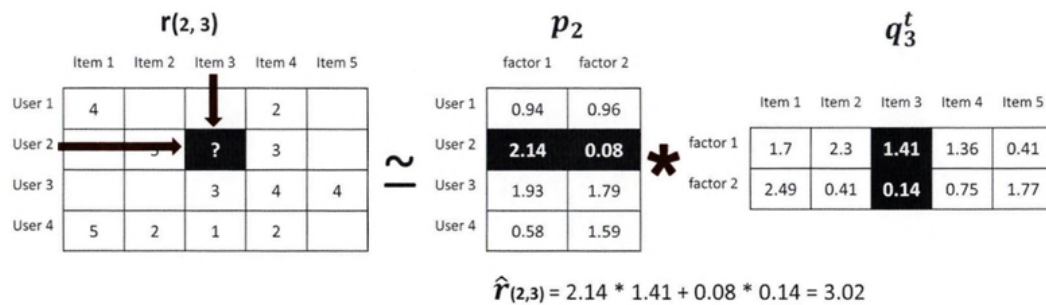


R행렬의 u행 사용자와 i열 아이템 위치에 있는 평점 데이터: $r(u,i)$

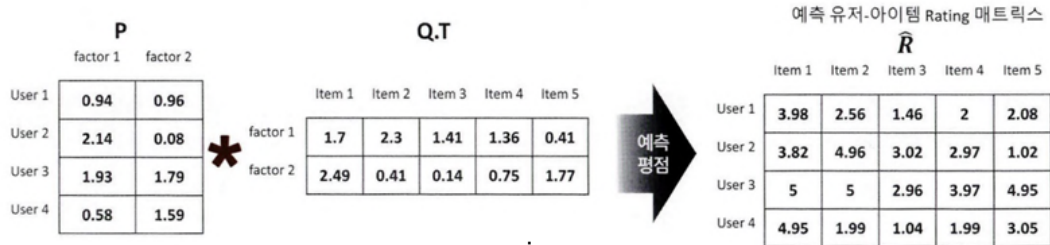
$$r_{u,i} = p_u * q_i^t$$



사용자가 평가하지 않은 아이템에 대한 평점도 잠재요인으로 분해된 P행렬과 Q 행렬을 이용해 예측 가능 →



$$R \cong \hat{R} = P * Q.T$$



행렬 분해는 주로 SVD 방식 사용 but, SVD는 널 값이 없는 행렬에만 적용 가능

⇒ 확률적 경사 하강법(SGD)이나 ALS(Alternating Least Squares) 방식을 이용해 SVD 수행

확률적 경사 하강법을 이용한 행렬 분해

:P와 Q 행렬로 계산된 예측 R 행렬 값이 실제 R 행렬 값과 가장 최소의 오류를 가질 수 있도록 반복적인 비용 함수 최적화를 통해 P와 Q를 유추해내는 것

[과정]

1. P와 Q를 임의의 값을 가진 행렬로 설정
2. P와 Q.T 값을 곱해 예측 R 행렬을 계산하고 예측 R 행렬과 실제 R 행렬에 해당하는 오류 값을 계산
3. 이 오류 값을 최소화할 수 있도록 P와 Q행렬을 적절한 값으로 각각 업데이트
4. 만족할만한 오류 값을 가질 때까지 2,3번 작업을 반복하면서 P와 Q 값을 업데이트해 근사화

$$\min \sum (r_{u,i} - p_u q_i^t)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2)$$

⇒ 실제 값과 예측값의 오류 최소화과 L2 규제를 고려한 비용 함수식

$$\begin{aligned}\dot{p}_u &= p_u + \eta (e_{(u,i)} * q_i - \lambda * p_u) \\ \dot{q}_i &= q_i + \eta (e_{(u,i)} * p_u - \lambda * q_i)\end{aligned}$$

⇒ 비용 함수를 최소화하기 위해 새롭게 업데이트되는 $P'u$, $q'i$

- $P'u$: P 행렬의 사용자 u행 벡터
- $q'i$: Q 행렬의 아이템 i행의 전치 벡터
- $r(u,i)$: 실제 R 행렬의 u행, i열에 위치한 값
- $\hat{r}(u,i)$: 예측 $\hat{R}$ 행렬의 u행, i열에 위치한 값 ($P_u * q'_i$ 로 계산)
- $e(u,i)$: u행, i열에 위치한 실제 행렬 값과 예측 행렬 값의 차이 오류. $r(u,i) - \hat{r}(u,i)$
- η : SGD 학습률
- λ : L2 규제 계수

L2 규제를 반영해 실제 R 행렬 값과 예측 R 행렬 값의 차이를 최소화하는 방향성을 가지고 P행렬과 Q행렬에 업데이트 값을 반복적으로 수행하면서 최적화된 예측 R 행렬을 구하는 방식이 SGD 기반의 행렬 분해

08. 파이썬 추천 시스템 패키지-Surprise

Surprise 패키지 소개

: 파이썬 기반의 추천 시스템 구축을 위한 전용 패키지

: 사이킷런과 유사한 API와 프레임워크를 제공

- 다양한 추천 알고리즘. 예를 들어 사용자 또는 아이템 기반 최근접 이웃 협업 필터링, SVD, SVD++, NMF 기반의 잠재 요인 협업 필터링을 쉽게 적용해 추천 시스템 구축
- Surprise의 핵심 API는 사이킷런의 핵심 API와 유사한 API 명으로 작성됨.
 - `fit()`, `predict()` API로 추천 데이터 학습과 예측, `train_test_split()`으로 추천 학습 데이터 세트와 예측 데이터 세트 분리, `cross_validate()`, `GridSearchCV` 클래스를 통해 추천

시스템을 위한 모델 셀렉션, 평가, 하이퍼 파라미터 튜닝 등의 기능 제공

Surprise 주요 모듈 소개

Dataset

Surprise는 userjd (사용자 아이디) , item_id (아이템 아이디) , rating (평점) 데이터가로우 레벨로 된 데이터 세트만 적용 ⇒ 네번째 칼럼부터는 로딩 수행X

API 명	내용
Dataset.load_builtin (name='ml-100k')	무비렌즈 아카이브 FTP 서버에서 무비렌즈 데이터를 내려받습니다. ml-100k, ml-1M를 내려받을 수 있습니다. 일단 내려받은 데이터는 .surprise_data 디렉터리 밑에 저장되고, 해당 디렉터리에 데이터가 있으면 FTP에서 내려받지 않고 해당 데이터를 이용합니다. 입력 파라미터인 name으로 대상 데이터가 ml-100k인지 ml-1m인지를 입력합니다(name='ml-100k'). 디폴트는 ml-100k입니다.

API 명	내용
Dataset.load_from_file (file_path, reader)	OS 파일에서 데이터를 로딩할 때 사용합니다. coma, 탭 등으로 칼럼이 분리된 포맷의 OS 파일에서 데이터를 로딩합니다. 입력 파라미터로 OS 파일명, Reader로 파일의 포맷을 지정합니다.
Dataset.load_from_df (df, reader)	판다스의 DataFrame에서 데이터를 로딩합니다. 파라미터로 DataFrame을 입력받으며 DataFrame 역시 반드시 3개의 칼럼인 사용자 아이디, 아이템 아이디, 평점 순으로 칼럼 순서가 정해져 있어야 합니다. 입력 파라미터로 DataFrame 객체, Reader로 파일의 포맷을 지정합니다.

Surprise 추천 알고리즘 클래스

클래스명	설명
SVD	행렬 분해를 통한 잠재 요인 협업 필터링을 위한 SVD 알고리즘.
KNNBasic	최근접 이웃 협업 필터링을 위한 KNN 알고리즘.
BaselineOnly	사용자 Bias와 아이템 Bias를 감안한 SGD 베이스라인 알고리즘.

Surprise SVD의 비용 함수는 사용자 베이스라인 편향성을 감안한 평점 예측에 Regularization 적용

- 사용자 예측 Rating: $\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u$
- Regularization을 적용한 비용 함수: $\sum (r_{ui} - \hat{r}_{ui})^2 + \lambda (b_u^2 + b_i^2 + \|q_i\|^2 + \|p_u\|^2)$

[SVD 클래스의 입력 파라미터]

파라미터명	내용
n_factors	잠재 요인 K의 개수. 디폴트는 100. 커질수록 정확도가 높아질 수 있으나 과적합 문제가 발생할 수 있습니다.
n_epochs	SGD(Stochastic Gradient Descent) 수행 시 반복 횟수. 디폴트는 20.
biased (bool)	베이스라인 사용자 편향 적용 여부이며, 디폴트는 True입니다.

[추천 알고리즘의 예측 성능 벤치마크 결과]

알고리즘 유형	RMSE	MAE	Time
SVD	0.934	0.737	0:00:11
SVD++	0.92	0.722	0:09:03
NMF	0.963	0.758	0:00:15
Slope One	0.946	0.743	0:00:08
k-NN	0.98	0.774	0:00:10
Centered k-NN	0.951	0.749	0:00:10
k-NN Baseline	0.931	0.733	0:00:12
Co-Clustering	0.963	0.753	0:00:03
Baseline	0.944	0.748	0:00:01

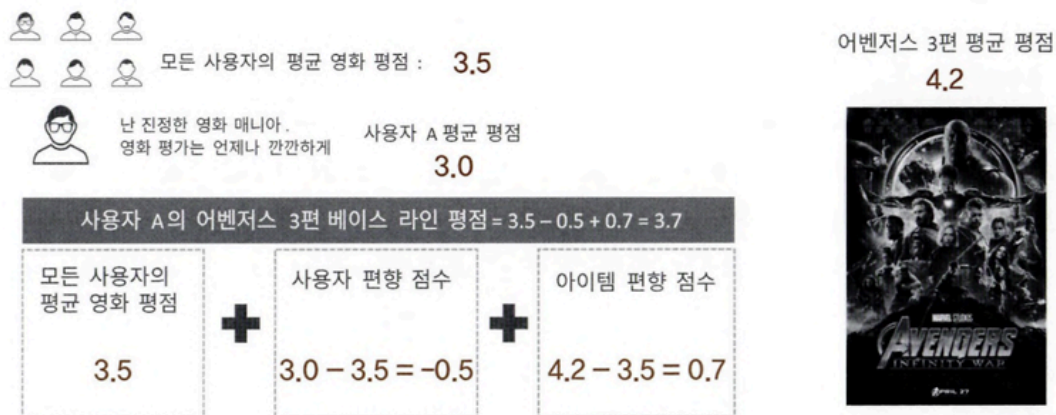
- SVD++ : RMSE, MAE 성적이 가장 좋음 but, 상대적으로 시간이 오래걸림(데이터 커지면 어려움)
- SVD와 K-NN Baseline이 가장 성능 평가 수치가 좋음
- k-NN 자체는 성능이 뒤쳐지지만, Baseline결합하면 수치 향상

- Baseline: 각 개인이 평점을 부여하는 성향을 반영해 평점을 계산

베이스라인 평점

: 개인의 성향에 따라서 점수를 부여하는 것을 반영(bias 요소 반영)해 평점을 부과하는 것

- 보통 베이스라인 평점: 전체 평균 평점 + 사용자 편향 점수 + 아이템 편향 점수
 - 전체 평균 평점 = 모든 사용자의 아이템에 대한 평점을 평균한 값
 - 사용자 편향 점수 = 사용자별 아이템 평점 평균 값 - 전체 평균 평점
 - 아이템 편향 점수 = 아이템별 평점 평균 값 - 전체 평균 평점



지원 파일

<https://colab.research.google.com/drive/1hebSH1FVWR5oPHg8XbADIfm4DdBvxtBH?usp=sharing>